

Create Tables in Hive

A. Customer Table

```
CREATE TABLE customers (  
    cust_id INT,  
    fname STRING,  
    lname STRING,  
    age INT,  
    profession STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE;
```

B. Transaction Table

```
CREATE TABLE transactions (  
    txn_id STRING,  
    txn_date STRING,  
    cust_id INT,  
    amount FLOAT,  
    category STRING,  
    product STRING,  
    city STRING,  
    state STRING,  
    payment_mode STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE;
```

Load Data

```
LOAD DATA Local INPATH '/home/cdacoctnov012/custs.txt' INTO TABLE customers;  
LOAD DATA Local INPATH '/home/cdacoctnov012/txns1.txt' INTO TABLE transactions;
```

A. Inner Join

To get all customers who made a transaction:

```
SELECT c.cust_id, c.fname, c.lname, t.amount FROM customers c
```

```
JOIN transactions t  
ON c.cust_id = t.cust_id limit 10;
```

➡ Returns only matching records between both tables.

Explanation

♦ Step 1: What the query does

The query joins two tables:

customers (alias c)

transactions (alias t)

On the condition `c.cust_id = t.cust_id`,

and then selects columns from both tables for the first 10 records.

♦ Step 2: Hive Query Execution Plan Overview

Hive translates this SQL into a MapReduce job (as shown in your EXPLAIN output).

The stages show how data flows through the system.

Stage Dependencies

Stage-1 is a root stage

Stage-0 depends on Stage-1

Stage-1 → runs the MapReduce join

Stage-0 → fetches the final 10 rows for output

: Stage-1 – The MapReduce Phase

Hive performs the join using MapReduce.

a) Map Phase

There are two TableScan operators (for customers and transactions):

For customers (alias c):

Reads all rows from the customers table.

Filters out rows where cust_id is NULL.

Selects only required columns: cust_id, fname, lname.

Prepares key-value pairs:

Key → cust_id

Value → other selected columns.

For transactions (alias t):

Reads all rows from the transactions table.

Filters out rows where cust_id is NULL.

Selects only cust_id and amount.

Also emits:

Key → cust_id

Value → amount.

So, both mappers emit the same join key: cust_id.

b) Shuffle & Sort

The MapReduce framework:

Partitions data by cust_id

Sends all rows with same cust_id to the same reducer

Sorts data on the join key

c) Reduce Phase (Join Operator)

The Reducer receives matching records from both tables.

Performs an INNER JOIN between customers (tag 0) and transactions (tag 1).

Produces combined output columns:

_col0 = cust_id

_col1 = fname

_col2 = lname

_col4 = amount

◆ Step 4: Stage-0 – The Fetch Phase

After the join is complete:

Hive applies the LIMIT 10

Fetches the first 10 joined results

Writes output using the SequenceFileOutputFormat

◆ Step 5: Statistics & Optimization Notes

Statistics: Num rows: 1 Data size: 372 Basic stats: COMPLETE Column stats: NONE

These are metadata estimates, not actual row counts.

Hive uses them for cost-based optimization (CBO).

Since both tables have small data (1 row estimate here), Hive likely uses common join (not map-side join).

Map-Side vs Reduce-Side Joins

Concept Recap

Join Type	When Used	Mechanism	Performance
Reduce-Side Join	Default (both large tables)	Data shuffled to reducers	Slow
Map-Side Join (Broadcast Join)	One small, one large table	Small table loaded in mapper memory	Fast

♦ A. Reduce-Side Join (Default)

When both tables are large:

```
SELECT c.fname, c.profession, t.category, t.amount
FROM customers c
JOIN transactions t
ON c.cust_id = t.cust_id;
```

Execution:

- Mapper emits key-value pairs by cust_id.
- Hive shuffles data by key to reducers.

- Reducers perform joins.
-

♦ B. Map-Side Join

What is a Map-Side Join?

A **map-side join** (also called **broadcast join**) is a type of join in Hive that **avoids the Reduce phase** entirely.

Instead of sending all data through the shuffle and sort (which is slow), Hive loads **the smaller table into memory on each mapper**, and the join happens **directly in the map phase**.

So:

- No data shuffle
 - No reduce phase
 - Much faster execution
-

♦ When to Use Map-Side Join

Use **map-side join** when one of the joining tables is **small enough to fit into memory** on each mapper.

Typical Scenarios:

1. **One small table, one large table**

- Example: customers (1,000 rows) joined with transactions (millions of rows).
- The small table (customers) is broadcast to all mappers.
- Each mapper reads part of the large table and joins with the small one in memory.

```
SELECT /*+ MAPJOIN(customers) */  
  c.cust_id, c.fname, c.lname, t.amount  
FROM customers c  
JOIN transactions t  
ON c.cust_id = t.cust_id limit 10;
```

2. Lookup joins

- When you use a small reference table (e.g., state codes, category names, etc.).
- Perfect for dimension–fact joins.

3. Frequent joins with a small dimension

- E.g., joining sales (fact) table with small product or region tables repeatedly.

♦ When NOT to Use Map-Side Join

Avoid it when:

1. Both tables are **large**
→ All mappers would try to load the smaller table into memory — which can cause **OutOfMemory errors**.
2. The small table is **not really small** (hundreds of MBs or more).
3. When **the optimizer cannot determine** table sizes correctly (if stats are missing), it might misbehave.

♦ Hive Configurations for Map-Side Join

Hive automatically tries to convert a join into a map-side join **if it detects a small table**, but you can control it:

Parameter	Description	Default
hive.auto.convert.join	Enables automatic map-side joins	true
hive.mapjoin.smalltable.filesize	Max size of the small table (bytes)	25 MB (default)
hive.auto.convert.join.noconditionaltask	Forces map join when applicable	false

You can override these in Hive CLI:

```
SET hive.auto.convert.join=true;
```

```
SET hive.mapjoin.smalltable.filesize=104857600; -- 100 MB
```

If you enable cost-based optimization and statistics (ANALYZE TABLE), Hive can **automatically pick map-side joins** when suitable.

```
ANALYZE TABLE customers COMPUTE STATISTICS;
```

```
ANALYZE TABLE transactions COMPUTE STATISTICS;
```

Then Hive decides internally whether to use map-side join or not.

Internal, External, and Managed Tables

Type	Data Storage	DROP Behavior	Use Case
Managed (Internal)	Hive Warehouse (default)	Deletes data + metadata	Temporary / Derived tables
External	User-specified HDFS path	Deletes only metadata	Shared data / permanent HDFS data

Example:

Managed Table

```
➡ Drop it: CREATE TABLE managed_customers (  
    cust_id INT, fname STRING, lname STRING, age INT, profession STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE;  
LOAD DATA INPATH '/user/cdacoctnov012/db/custs.txt' INTO TABLE managed_customers;
```



```
DROP TABLE managed_customers;
```

Both table and HDFS data deleted (/warehouse/retail_db.db/managed_customers removed).

External Table

```
CREATE EXTERNAL TABLE ext_customers (  
    cust_id INT, fname STRING, lname STRING, age INT, profession STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE  
LOCATION '/user/cdacoctnov012/db';
```

➡ Drop it:

```
DROP TABLE ext_customers;
```

Only metadata dropped — data in /external/customers remains.

What an External Table Means

An **external table** in Hive:

- Only stores **metadata** in Hive Metastore.
- The actual data **remains in HDFS** at the specified location.
- Dropping or altering the table **does not delete the data file**.

Can You Delete Records?

Operation	Internal Table	External Table
-----------	----------------	----------------

DELETE FROM table WHERE ...	(if transactional table)	(if transactional external table)
--------------------------------	----------------------------------	---

TRUNCATE TABLE	Yes	(not allowed for external tables)
----------------	-----	-----------------------------------

DROP TABLE	Deletes data & metadata	Deletes metadata only
------------	-------------------------	-----------------------

INSERT OVERWRITE	Yes	(can overwrite the file contents)
------------------	-----	-----------------------------------

If You're Using a Regular External Table (Non-Transactional)

You **cannot delete individual rows** because Hive reads the file as a whole.

But you can simulate deletion by:

```
CREATE TABLE new_table AS
```

```
SELECT * FROM ext_customers WHERE cust_id != 101;
```

Then replace the old data file if needed.

Then create the table like:

```
CREATE EXTERNAL TABLE ext_customers (
```

```
  cust_id INT,
```

```
  fname STRING,
```

```
  lname STRING,
```

```
age INT,  
profession STRING  
)  
  
CLUSTERED BY (cust_id) INTO 2 BUCKETS  
  
STORED AS ORC  
  
TBLPROPERTIES ('transactional'='true', 'external.table.purge'='true');
```

Now you can safely do:

```
DELETE FROM ext_customers WHERE age < 25;
```

♦ Step 7: ORC vs Parquet

Feature	ORC	Parquet
Storage Type	Columnar	Columnar
Compression	Snappy, Zlib	Snappy, Gzip
Hive Compatibility	Native (ACID supported)	Excellent via Spark
Best For	Hive analytical workloads	Cross-tool analytics (Spark, Presto, Impala)

Example: Create in ORC and Parquet

ORC Table

```
CREATE TABLE txns_orc  
STORED AS ORC  
AS SELECT * FROM transactions;
```

Parquet Table

```
CREATE TABLE txns_parquet  
STORED AS PARQUET  
AS SELECT * FROM transactions;
```

➡ Compare size:

```
hdfs dfs -du -h /user/hive/warehouse/nishad_one.db/txns_orc/
```

```
hdfs dfs -du -h /user/hive/warehouse/nishad_one.db/txns_parquet/
```

```
hdfs dfs -du -h /user/hive/warehouse/nishad_one.db/transactions
```

➡ Compare performance:

```
SELECT COUNT(*) FROM txns_orc;  
SELECT COUNT(*) FROM txns_parquet;
```

ORC usually smaller and faster in Hive; Parquet interoperable with Spark.

How TEXTFILE Stores It

A **TextFile** or **CSV** table will store it **row by row**:

```
1,Rahul,25,India  
2,Meena,30,USA  
3,Aditya,27,India  
4,John,35,UK  
5,Priya,22,India
```

Every query reads all fields — even if you just want country.

How ORC Stores It (Columnar Format)

In ORC, data is stored **column-wise** like this:

```
Column 1 (cust_id): [1, 2, 3, 4, 5]  
Column 2 (name):   [Rahul, Meena, Aditya, John, Priya]
```

Column 3 (age): [25, 30, 27, 35, 22]

Column 4 (country): [India, USA, India, UK, India]

Each **column** is stored **together**, not each row.

This allows compression algorithms (like ZLIB or Snappy) to compress similar data **very efficiently**.

When you query only `SELECT country FROM customers`, Hive reads only the country column blocks — skipping all others.

ORC File Internal Structure

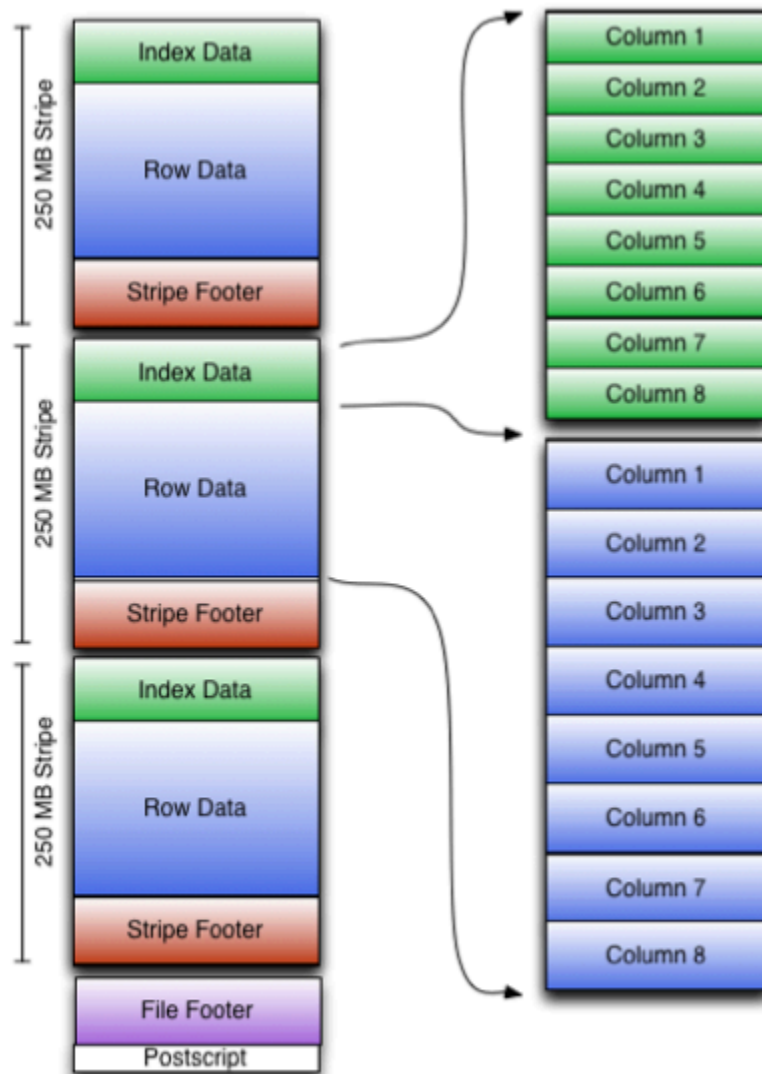
ORC file is divided into **stripes**, and each stripe has:

1. **Index Data** — min, max, and row positions for quick filtering
2. **Row Data** — actual encoded & compressed column data
3. **Stripe Footer** — metadata for that stripe

And the file ends with:

- **File Footer** (metadata about stripes, column types)

- **Postscript** (compression info, file length, etc.)



Each **stripe** might hold, say, 250 MB of data (configurable).

Example of How It Optimizes a Query

Let's say you run:

```
SELECT cust_id, country FROM customers WHERE country = 'India';
```

Hive + ORC will:

- Use the **column index** to find which stripes have country = 'India'
- **Skip reading entire stripes** where country ≠ India
- Only decompress and read cust_id and country columns

What is Parquet?

Apache Parquet is a **columnar storage format**, like ORC — but it's designed for **cross-platform** use (Hive, Spark, Impala, Presto, Pandas, etc.).

It's optimized for:

- Columnar compression
- Efficient encoding
- Predicate pushdown (filtering without reading full data)

Example Data

Let's use the same dataset

cust_id	name	age	country
1	Rahul	25	India
2	Meena	30	USA
3	Aditya	27	India
4	John	35	UK
5	Priya	22	India

How TextFile Stores It

Just like before, **row-wise**:

1,Rahul,25,India
2,Meena,30,USA
3,Aditya,27,India

4,John,35,UK
5,Priya,22,India

Every query reads every column.

How Parquet Stores It (Columnar Format)

Parquet also stores data **column by column**, but it adds a **nested structure** concept (great for JSON-like or complex data).

Internally:

Column 1 (cust_id): [1, 2, 3, 4, 5]

Column 2 (name): [Rahul, Meena, Aditya, John, Priya]

Column 3 (age): [25, 30, 27, 35, 22]

Column 4 (country): [India, USA, India, UK, India]

Each column is stored together (not row by row)

Data is divided into **row groups**, and each row group has **column chunks**

Each column chunk can use its own **encoding** (e.g., dictionary, delta encoding)

Parquet File Structure

Each **Row Group** can store, for example, **1 million rows**.

Each column chunk inside the row group is **compressed independently**.

Parquet File Example (Internal Layout)

Each file roughly has:

+-----+

| Row Group 1 |

| cust_id column chunk |

| name column chunk |


```

| age column chunk      |
| country column chunk  |
+-----+
| Row Group 2          |
| ...                  |
+-----+
| File Footer (schema, stats, etc.) |
+-----+

```

Each column chunk stores:

- Encoded data pages
- Dictionary pages (if applicable)
- Column index + min/max stats

How Query Optimization Works

For query:

```
SELECT cust_id, country FROM customers_parquet WHERE country = 'India';
```

Hive + Parquet:

- Looks at the **column index** → finds only row groups where country = 'India'
- Reads only cust_id and country chunks
- Skips name and age completely

 → Very efficient on large datasets

Key Difference: ORC vs Parquet

Feature	ORC	Parquet
Best for	Hive ecosystem	Cross-platform (Hive, Spark, Impala, etc.)
Compression	Slightly better (Zlib default)	Very good (Snappy default)
Metadata granularity	Stripe-level	Row-group level
ACID support	Yes	No (in Hive)
Schema evolution	Limited	More flexible
Read/Write performance	Faster reads	Faster writes
File extension	.orc	.parquet